AI 在金融中的應用

第八週 — 財務管理：波動率、風險與人工智慧

Yi-Hao Lai

2026 Spring

今日學習路徑

1. 故事：最終對決
2. ML vs. 計量經濟學：兩種哲學
3. 隨機森林用於波動率預測
4. LSTM 神經網路
5. Python 實作
6. 裁決：GARCH vs. ML
7. 金融科技前沿
8. 重點回顧

1

故事：最終對決

VolTech 的挑戰

情境：QuantStar 宣稱他們的 ML 系統比傳統模型好 **50%**。

Priya 設置了擂台：左邊螢幕是 GARCH，右邊螢幕是神經網路儀表板。

我們能驗證 — 或推翻 — 這個說法嗎？

核心問題

在什麼條件下 ML 能勝過 GARCH？我們如何聰明地將兩者結合？

今日學習目標

在本節課結束後，你將能夠：

1. 解釋模型選擇中的偏差-變異取捨（Bias-Variance Tradeoff）
2. 使用工程特徵建構隨機森林（Random Forest）波動率模型
3. 理解 LSTM 神經網路的架構與閘門機制
4. 使用 RMSE、MAE 和 QLIKE 比較 ML vs. GARCH
5. 設計 GARCH + ML 混合模型

2

ML vs. 計量經濟學：兩種哲學

波動率建模的兩種方法

GARCH（計量經濟學）

- 從**理論**出發
- 5–7 個可解讀的參數
- 強假設（參數式）
- 少量資料即可運作
- 可審計且符合 Basel 規範

ML（資料驅動）

- 從**資料**出發
- 數千個參數
- 彈性大（非參數式）
- 需要大量資料集
- 通常是「黑箱」

⚠ 關鍵洞見

兩者都不是絕對優越的。選擇取決於資料狀態、預測期間以及可解讀性的要求。

偏差-變異取捨（Bias-Variance Tradeoff）

預測誤差分解

$$\text{MSE} = \underbrace{\text{Bias}^2}_{\text{模型僵化}} + \underbrace{\text{Variance}}_{\text{模型不穩定}} + \underbrace{\text{Noise}}_{\text{不可約減}}$$

	偏差	變異
GARCH	高（假設僵化）	低（穩定）
神經網路	低（彈性大）	高（不穩定）

在嘈雜的金融資料中：高偏差/低變異的模型在短期通常勝出。ML 的優勢在特徵更豐富、期間更長時才會浮現。

3

隨機森林用於波動率預測

隨機森林（Random Forest）：集成方法

隨機森林在資料和特徵的隨機子集上建構許多決策樹。

為什麼有效：

- 捕捉非線性模式
- 對離群值穩健
- 內建特徵重要性排序
- 不需分配假設

預測

$\hat{y} = \frac{1}{B}\sum_{b=1}^{B} T_b(\mathbf{x})$

B 棵樹的平均值降低變異，但不增加偏差。

波動率的特徵工程（Feature Engineering）

特徵類型	範例
落後報酬率	$r_{t-1}, r_{t-2}, \ldots, r_{t-5}$
落後平方報酬率	$r_{t-1}^2, \ldots, r_{t-5}^2$ （≈ ARCH 項）
滾動波動率	5 天、10 天、22 天滾動 σ （≈ β 項）
絕對報酬率	$\lvert r_{t-1}\rvert, \ldots, \lvert r_{t-5}\rvert$
GARCH 變異數	$\hat{\sigma}_t^2$ 來自 GJR-GARCH （混合特徵 !）

關鍵洞見：最重要的 ML 特徵恰好反映了 GARCH 的結構。隨機森林正在從資料中重新發現波動率群聚。

4

LSTM 神經網路

LSTM：時間序列的記憶

長短期記憶網路（Long Short-Term Memory）逐步處理序列資料，維護一個「記憶細胞」來記錄過去的模式。

三個閘門：

1. **遺忘閘門（Forget Gate）**：「我應該丟棄舊的波動率資訊嗎？」（市場狀態轉換）
2. **輸入閘門（Input Gate）**：「今天的報酬率是否重要到值得記住？」（衝擊偵測）
3. **輸出閘門（Output Gate）**：「根據我記住的內容，我的波動率預測是什麼？」

問題所在

LSTM 可以自行學習類似 GARCH 的行為，但 GARCH 只用 **5 個參數**就能做到。LSTM 需要**數千個**參數和更多的資料。

LSTM 閘門方程式

閘門機制（簡化版）

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad \text{遺忘閘門}$$
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad \text{輸入閘門}$$
$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad \text{輸出閘門}$$

- σ = sigmoid 函數（輸出介於 0 和 1 之間）
- h_{t-1} = 前一隱藏狀態（過去的摘要）
- x_t = 當前輸入（今天的報酬率）
- 每個閘門從資料中學習何時該動作

5

Python 實作

步驟一：特徵工程

```python
import numpy as np, pandas as pd
import yfinance as yf
from sklearn.ensemble import (
    RandomForestRegressor)
from arch import arch_model

sp500 = yf.download("^GSPC",
    start="2010-01-01", end="2024-12-31")
returns = 100 * np.log(
    sp500["Close"] / sp500["Close"].shift(1)
).dropna().squeeze()
target = returns.shift(-1) ** 2
```

步驟二：建構特徵矩陣

```python
features = pd.DataFrame(index=returns.index)
for lag in range(1, 6):
    features[f"ret_lag{lag}"] = (
        returns.shift(lag))
    features[f"ret2_lag{lag}"] = (
        returns.shift(lag) ** 2)
for w in [5, 10, 22]:
    features[f"rvol_{w}d"] = (
        returns.rolling(w).std())
for lag in range(1, 4):
    features[f"abs_ret_lag{lag}"] = (
        returns.shift(lag).abs())
```

結果：18 個工程特徵，捕捉波動率動態

步驟三：時間序列分割

```python
data = pd.concat([features,
    target.rename("target")], axis=1).dropna()
split = "2020-01-01"
train = data[data.index < split]
test = data[data.index >= split]

X_train = train.drop("target", axis=1)
y_train = train["target"]
X_test = test.drop("target", axis=1)
y_test = test["target"]
```

關鍵

時間序列資料絕對不能隨機打亂！必須按日期分割，以避免前瞻偏差（Look-Ahead Bias）。

步驟四：隨機森林模型

```python
rf = RandomForestRegressor(
    n_estimators=500, max_depth=10,
    min_samples_leaf=20,
    max_features="sqrt", random_state=42)
rf.fit(X_train, y_train)
rf_pred = pd.Series(rf.predict(X_test),
    index=X_test.index)

importance = pd.Series(
    rf.feature_importances_,
    index=X_train.columns
).sort_values(ascending=False)
print("Top 5:", importance.head(5))
```

步驟五：混合模型（Hybrid Model）

```python
# Add GARCH variance as a feature
model = arch_model(returns, vol="Garch",
    p=1, o=1, q=1, dist="t")
res = model.fit(last_obs=split, disp="off")
garch_var = res.conditional_volatility ** 2

data_h = data.copy()
data_h["garch_var"] = garch_var
# ... re-split and train hybrid RF ...
```

混合模型的秘訣：讓 GARCH 將其結構性知識濃縮成一個特徵，然後讓 ML 在此基礎上學習。

步驟六：正面對決比較

```python
def evaluate(actual, forecast, name):
    a, f = actual.values, np.maximum(
        forecast.values, 1e-10)
    rmse = np.sqrt(np.mean((a - f) ** 2))
    mae = np.mean(np.abs(a - f))
    qlike = np.mean(np.log(f) + a / f)
    return {"Model": name, "RMSE": rmse,
            "MAE": mae, "QLIKE": qlike}
```

預期結果：混合模型 > RF > GARCH（大多數指標）。但 GARCH 可能在 QLIKE 上勝出，因為它在平靜期間的校準更好。

6

裁決：GARCH vs. ML

正面對決結果

評估標準	GJR-GARCH	隨機森林	混合模型
1 天 RMSE	5.61	5.52	**5.21**
1 天 QLIKE	3.65	3.82	**3.51**
可解讀性	高	低	中等
訓練速度	快	中等	中等
監管適配性	強	弱	中等
危機反應	好	不穩定	好

QuantStar 所謂的「提升 50%」是基於樣本內結果，且存在前瞻偏差。

三大關鍵發現

1. **GARCH 在日頻預測上極具競爭力**
   僅用 5–7 個參數，即可捕捉主導性的波動率動態。單獨使用 ML 幾乎無法超越它。

2. **混合模型一致性地勝出**
   GARCH 作為特徵 + ML 的彈性 = 最佳預測。整體大於部分之和。

3. **可解讀性具有經濟價值**
   一個監管機構能審計的模型，可能帶來比更準確的黑箱更好的商業成果。

Alex 給 Kenji 的建議

最終裁決

部署 GJR-GARCH 作為**主要風險引擎**，用於每日 VaR 和 ES 報告 — 它快速、可解讀，且符合 Basel 規範。

將混合隨機森林作為**輔助監控工具**，用於標記 GARCH 可能低估風險的時刻。

兩全其美：監管合規與前沿分析兼得。

7

金融科技前沿

AI + 金融的下一步

1. **基礎模型（Foundation Models）**：LLM 用於情緒分析、法說會摘要、風險敘述
2. **可解釋 AI（Explainable AI, XAI）**：SHAP 值、注意力圖讓黑箱變透明
3. **即時風險管理**：串流 ML 管線處理逐筆交易資料 + 新聞 + 社群媒體
4. **民主化**：開源工具（`arch`、`scikit-learn`、`pytorch`）讓量化分析變得人人可及
5. **監管科技（RegTech）**：AI 自動化合規報告、異常偵測、稽核軌跡

能在職場中脫穎而出的專業人士，將是同時理解 金融理論和 *AI* 工具的人。

8

重點回顧

六大重點

1. **ML 和 GARCH 是互補的**：GARCH 提供結構；ML 增加彈性
2. **GARCH 在日頻難以被超越**：5–7 個參數就能捕捉主導性的動態
3. **特徵工程至為關鍵**：最重要的 ML 特徵恰好反映了 GARCH 自身的結構
4. **混合模型勝出**：GARCH 作為特徵，一致性地產出最佳預測
5. **可解讀性具有價值**：監管合規要求模型可解釋性
6. **未來是整合**：計量經濟學結構 + ML 彈性 + 人類判斷

任務八：VolTech AI 風險報告

交付項目

1. 建構 Nikkei 225 和 S&P 500 的隨機森林模型（15+ 個特徵）
2. 建構含 GJR-GARCH 特徵的混合模型
3. 使用 RMSE、MAE、QLIKE 比較 GARCH、RF 和混合模型
4. 建立預測比較視覺化圖表
5. 撰寫 500 字給 Kenji 委員會的最終備忘錄

加分題：實作簡單 LSTM 並與隨機森林比較

旅程回顧：八週總覽

週次	關鍵里程碑
1	發現厚尾與非常態性
2	學會衡量時變波動率
3	建構第一個 GARCH 模型
4	以 GJR-GARCH 捕捉非對稱性
5	建立嚴謹的預測評估框架
6	計算 VaR 與預期損失
7	撰寫符合監管要求的報告
8	將 AI 與傳統模型整合

從厚尾到 AI：你已從零開始建構了一套完整的風險管理系統。

結束 — 也是開始

尾聲

三個月後，Alex 坐在他的新角落辦公室 — **VolTech Analytics 資深量化分析師**。

混合 GARCH-ML 系統已上線，服務亞洲四家機構客戶。

新的挑戰來了：跳躍風險、市場傳染效應、逐筆交易資料。

量化金融的旅程永遠不會真正結束。

歡迎來到前沿。